

# Seeding Developer Libraries with Sample Members







# SEEDING DEVELOPER LIBRARIES



Part of provisioning developer libraries is to add default or sample members to them.

This helps new hires get started with their work and demonstrates company standards for program structure, naming conventions, and coding standards.

We can load members into libraries in batch mode using the z/OS utility IEBCOPY.

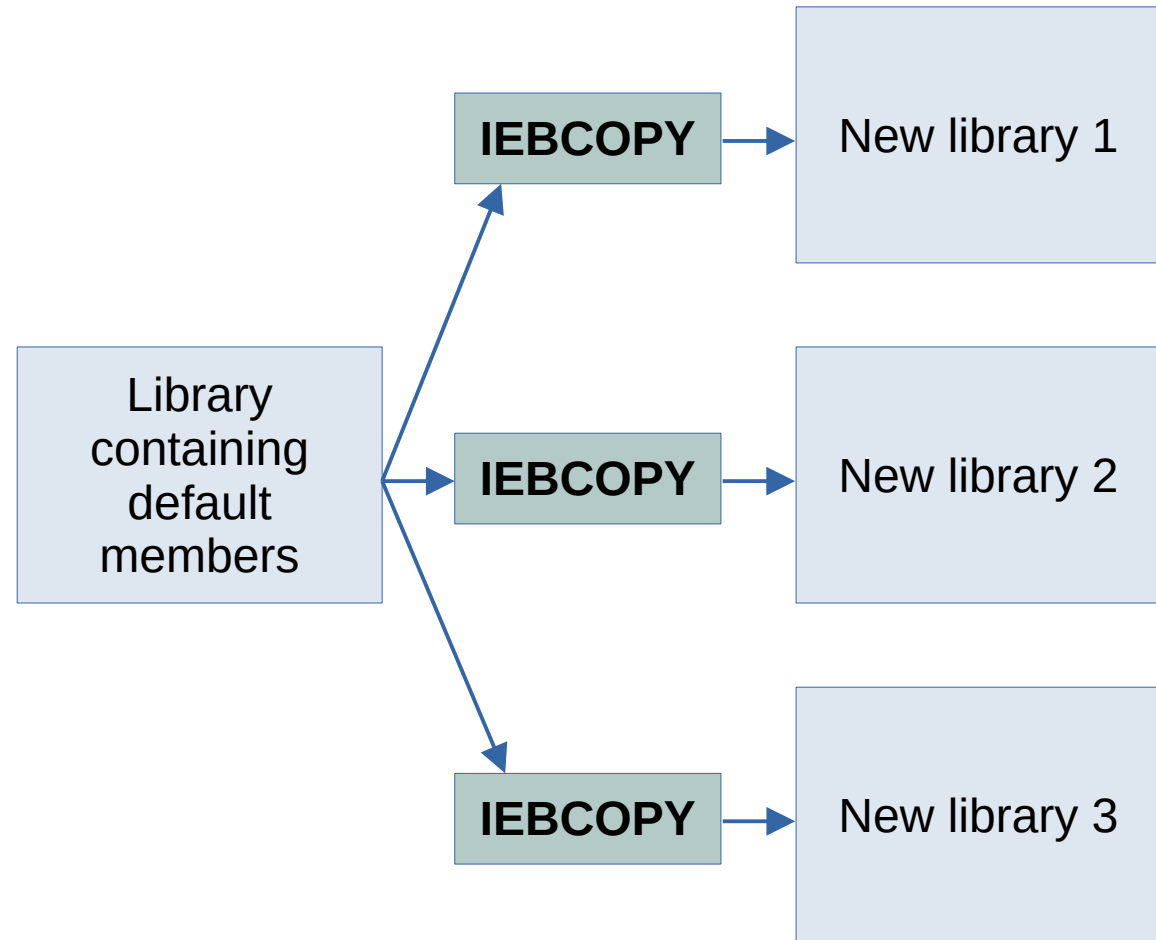
# SEEDING DEVELOPER LIBRARIES

There has to be a source from which to copy default members into new libraries. Creating that source is preliminary work we need to do before we can add this step to our provisioning process.

Since source libraries of all kinds have the same data set attributes, we can place all the sample members for all the source libraries in a single library.

If it makes more sense to separate the default members by category or according to some other scheme, we can define multiple libraries.

For instance, we might put all the default JCL members in one base library, and all the default COBOL source members in another. Any setup that is sensible in context will work.



# SEEDING DEVELOPER LIBRARIES, EXAMPLE

As an example, we've set up two "seed" libraries, one for JCL and one for Assembler source code.

The JCL seed library contains two members with sample jobs and one member with a JCL Procedure.

The ASM seed library contains a sample Assembler program named HELLO.

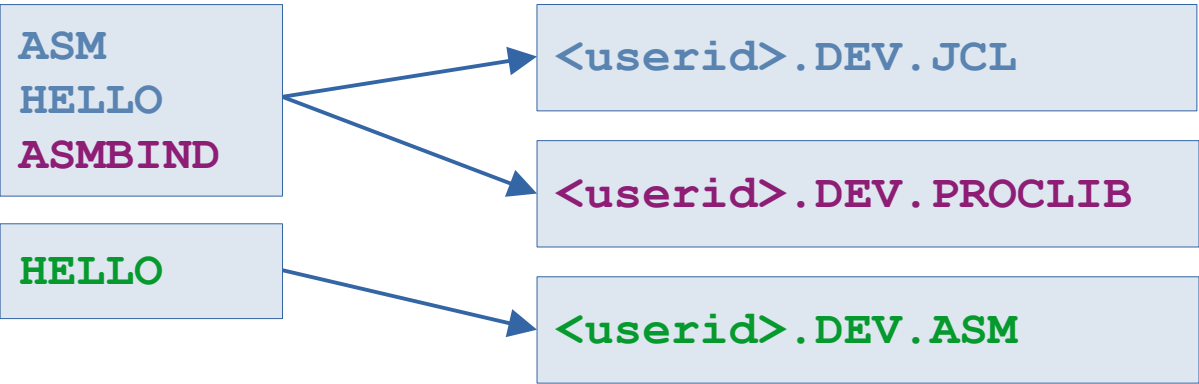
We want to use utility IEBCOPY to copy these members into each developer's "development" libraries. There's no need to seed the "test" libraries or the program libraries.

The example is small, but demonstrates how to copy members from one source library into multiple target libraries, and how to copy members from one source to one target library.

Some combination of those features of IEBCOPY can be used to seed all the libraries that need default members.

| <u>M</u> enu              | <u>F</u> unctions | <u>U</u> tilities | <u>H</u> elp |                     |                 |       |
|---------------------------|-------------------|-------------------|--------------|---------------------|-----------------|-------|
| EDIT IBMUSER.LAB.SEED.JCL |                   |                   |              | Member              | ASM             | saved |
| Command ==>               |                   |                   |              | Scroll              | ==> <u>PAGE</u> |       |
| Name                      | Prompt            | Size              | Created      | Changed             | ID              |       |
| . ASM                     |                   | 5                 | 2026/06/11   | 2026/06/11 10:09:56 | IBMUSER         |       |
| . ASMBIND                 |                   | 22                | 2026/06/11   | 2026/06/11 10:04:49 | IBMUSER         |       |
| . HELLO                   |                   | 7                 | 2026/06/11   | 2026/06/11 09:52:44 | IBMUSER         |       |
| **End**                   |                   |                   |              |                     |                 |       |

| <u>M</u> enu | <u>F</u> unctions    | <u>U</u> tilities | <u>H</u> elp           |                     |           |
|--------------|----------------------|-------------------|------------------------|---------------------|-----------|
| <hr/>        |                      |                   |                        |                     |           |
| EDIT         | IBMUSER.LAB.SEED.ASM |                   | Row 0000001 of 0000001 |                     |           |
| Command ==>  |                      |                   | Scroll ==> PAGE        |                     |           |
| <u>Name</u>  | <u>Prompt</u>        | <u>Size</u>       | <u>Created</u>         | <u>Changed</u>      | <u>ID</u> |
| . HELLO      |                      | 15                | 2026/06/11             | 2026/06/11 08:58:30 | IBMUSER   |
| **End**      |                      |                   |                        |                     |           |



# SEEDING DEVELOPER LIBRARIES, EXAMPLE

```

File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(SEEDLIBS) - 01.16      Columns 00001 00072
Command ==>                                     Scroll ==> CSR
***** Top of Data *****
000001 //IBMUERS  JOB , 'SEED LIBRARIES',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //SEEDJCL   EXEC PGM=IEBCOPY
000005 //SYSPRINT  DD SYSOUT=*
000006 //SYSOUT    DD SYSOUT=*
000007 //JCLSEED   DD DSN=&SYSUID..LAB.SEED.JCL,DISP=SHR
000008 //JCLOUT     DD DSN=&SYSUID..DEV.JCL,DISP=SHR
000009 //ASMSEED    DD DSN=&SYSUID..LAB.SEED.ASM,DISP=SHR
000010 //ASMOUT     DD DSN=&SYSUID..DEV.ASM,DISP=SHR
000011 //PROCLIB   DD DSN=&SYSUID..DEV.PROCLIB,DISP=SHR
000012 //SYSIN     DD *
000013 COPYOPER COPY OUTDD=JCLOUT,INDD=JCLSEED
000014          SELECT MEMBER=(ASM,HELLO)
000015 COPYOPER COPY OUTDD=PROCLIB,INDD=JCLSEED
000016          SELECT MEMBER=ASMBIND
000017 COPYOPER COPY OUTDD=ASMOUT,INDD=ASMSEED
000018          SELECT MEMBER=HELLO
000019 /*
***** Bottom of Data *****

```

ASM  
HELLO  
ASMBIND

HELLO

<userid>.DEV.JCL

<userid>.DEV.PROCLIB

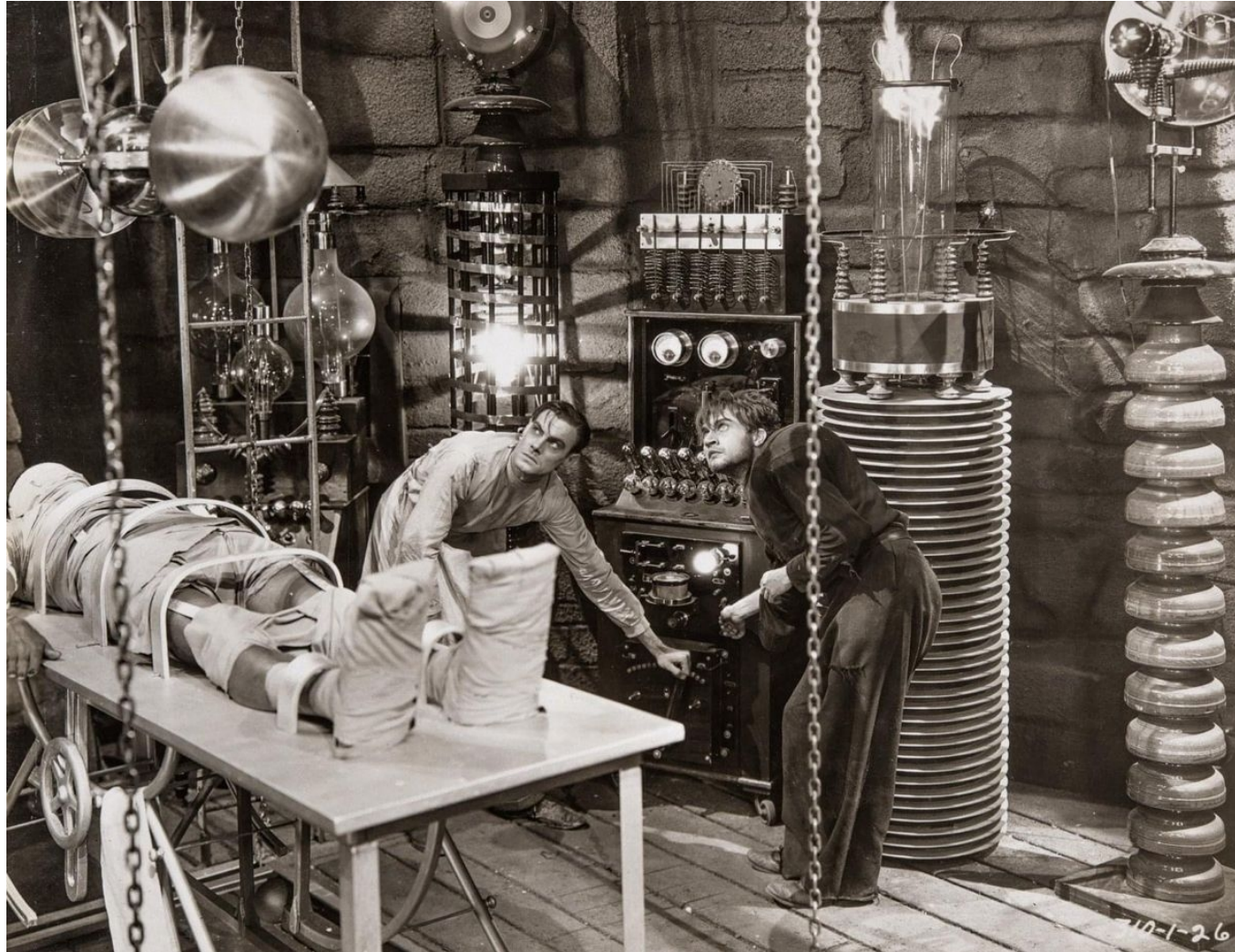
<userid>.DEV.ASM

This sample job copies members from the two seed libraries into the three target libraries shown in the diagram below. You can use JCL similar to this to copy members into the libraries you've provisioned for each application developer.

In the provisioning job stream you have now, there are no dependencies between the various individual jobs. When adding this functionality to the job stream, remember that you can't seed a library before you've allocated it. Job steps like this example have to be part of the same job that allocates each target library. Your solution may not look exactly like the example.



# LAB Z/OS 17: SEED SOME LIBRARIES



# CONDITIONAL JOB STEP EXECUTION

There is a lot more to JCL than what we've seen here, but you don't have to be an expert in JCL to do Platform Engineering work. Basic JCL skills are sufficient. There is one more detail that will be useful to know.

When a job step depends on the completion status of a previous step, we can control whether the step is executed based on the *condition code* set in the previous step (or all previous steps).

There are a couple of ways to do this, and one of them is shown in the sample JCL at right. It makes no sense to execute step SEEDJCL if any previous step in the job failed.

We can suppress its execution using the COND parameter on the EXEC statement. The value COND=(0,GT) means "if any previous step in this job returned with a condition code greater than zero, skip this step."

You can add this to your own JCL if you wish.

```

File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(PROVDEV) - 01.06      Columns 00001 00072
Command ==> Scroll ==> CSR
000001 //IBMUSER1 JOB , 'ALLOC SOURCE LIBS 1',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //          JCLLIB ORDER=(&SYSUID..LAB.PROCLIB)
000005 //*****
000006 /* ALLOCATE SOURCE LIBRARIES FOR JCL
000007 //*****
000008 //DEVJCL  EXEC MAKELIB,LIBNAME=&SYSUID..DEV.JCL,PRI=2,SEC=2
000009 //TSTJCL  EXEC MAKELIB,LIBNAME=&SYSUID..TST.JCL
000010 //DEVPROC EXEC MAKELIB,LIBNAME=&SYSUID..DEV.PROCLIB
000011 //TSTPROC EXEC MAKELIB,LIBNAME=&SYSUID..TST.PROCLIB
000012 //*****
000013 /* SEED DEV JCL SOURCE LIBRARY WITH DEFAULT MEMBERS
000014 //*****
000015 //SEEDJCL  EXEC PGM=IEBCOPY,COND=(0,GT)
000016 //SYSPRINT DD SYSOUT=*
000017 //SYSOUT   DD SYSOUT=*
000018 //JCLSEED  DD DSN=&SYSUID..LAB.SEED.JCL,DISP=SHR
000019 //JCLOUT   DD DSN=&SYSUID..DEV.JCL,DISP=SHR
000020 //PROCOUT  DD DSN=&SYSUID..DEV.PROCLIB,DISP=SHR
000021 //SYSIN    DD *
000022 COPYOPER  COPY OUTDD=JCLOUT,INDD=JCLSEED
000023          SELECT MEMBER=(ASM,HELLO)
000024 COPYOPER  COPY OUTDD=PROCOUT,INDD=JCLSEED
000025          SELECT MEMBER=(ASMBIND)
000026 /*

```



# USING TSO COMMANDS TO PROVISION LIBRARIES

```
alloc da(test.pds) new catalog space(1,1) dsntype(library) dsorg(po) recfm(f,b)
lrecl(80) blksize(27920)
READY
alloc da(test.pds2) like(test.pds)
READY
listds test.pds2
IBMUSER.TEST.PDS2
--RECFM-LRECL-BLKSIZE-DSORG
FB      80      32720    PO
--VOLUMES--
USRVS1
READY
alloc fi(infile) da(test.pds2(alpha))
READY
alloc fi(outfile) da(test.pds(alpha))
READY
ocopy indd(infile) outdd(outfile) text
READY
free file(infile)
READY
free file(outfile)
READY
```

You can do the same things using TSO commands as you have been doing using JCL. This is useful to know because you can choose to submit JCL or run TSO commands from off-platform tooling such as Ansible.

There will be situations when it's easier to use JCL and situations when it's easier to issue TSO commands.

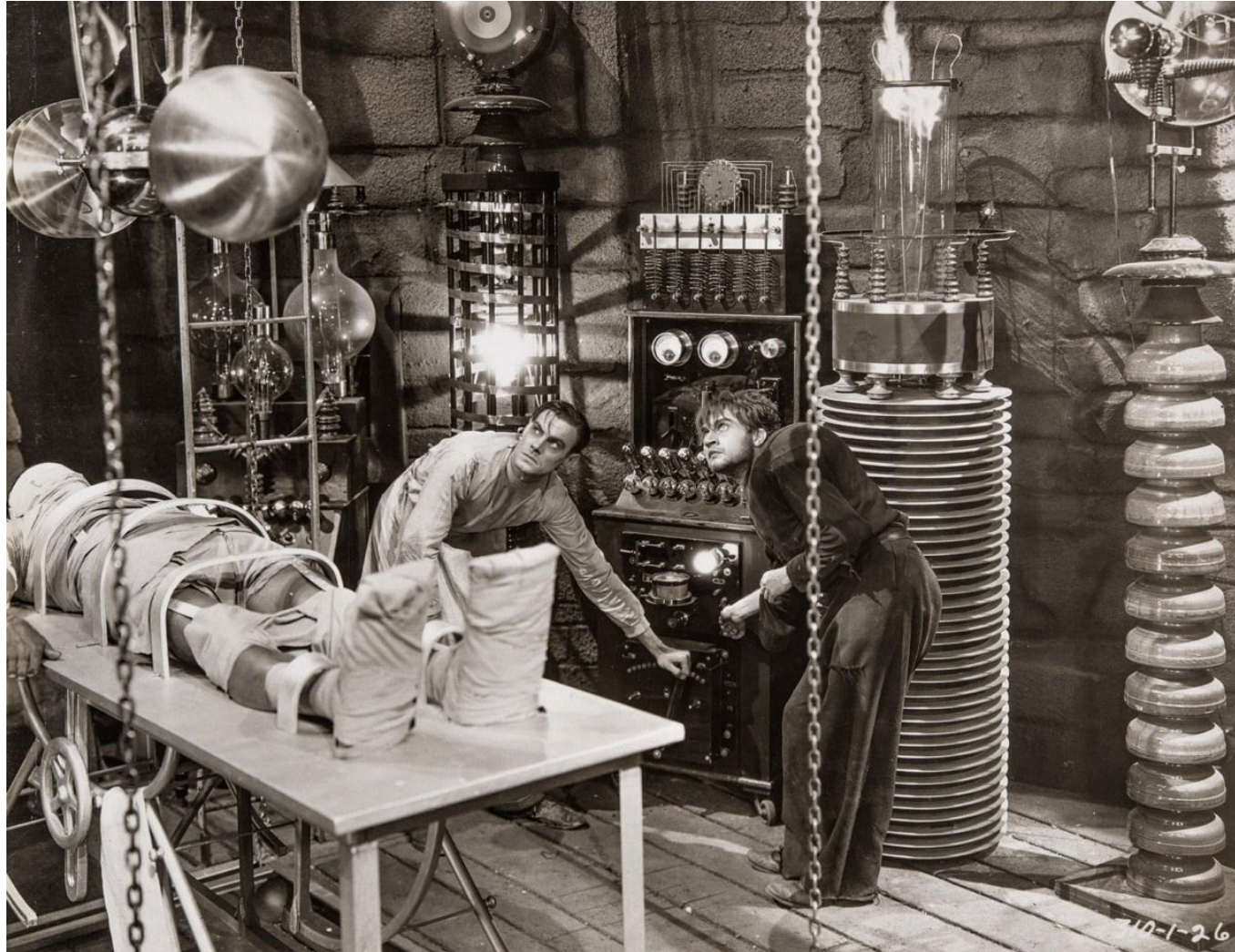
The examples at left show some TSO commands that you could use to provision libraries for an app developer.

The first **alloc** uses almost the same parameters as a DD statement in JCL to specify the attributes of a source library.

The second **alloc** creates another library based on the attributes of the first one.

After adding a member to test.pds2 (not shown here; I used ISPF), the **ocopy** command copies the member into the other library. You could use **ocopy** to place the member in the first pds as well, or upload it via a 3270 emulator, or copy it to the z/OS instance via Ansible or some other off-platform system management tool.

# LAB Z/OS 18: PRACTICE USING TSO COMMANDS



# WHAT YOU KNOW AND WHAT YOU CAN DO

- You can use the IEBCOPY utility to copy selected members from one or more source libraries into one or more target libraries.
- You can suppress the execution of a job step based on the completion code from one or more previous steps in the same job.
- You know that you can issue individual TSO commands to accomplish the same thing as your provisioning job.
- You know how to look for more information about these TSO commands.



# WHAT'S NEXT

- VSAM